Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase IV · Escala, sistemas y arquitecturas modernas

Módulo 19

Inferencia y generación

Decodificación, caché KV, decodificación especulativa, cuantización y servicio a escala

---

Resumen del módulo

Edición de octubre de 2026

Sobre este resumen

Este documento reúne lo indispensable de los apuntes del Módulo 19: el texto expositivo, todas las definiciones y todos los resultados (teoremas, proposiciones, lemas y corolarios), las tablas y las observaciones de contenido. Se han suprimido las demostraciones, los ejemplos, las lecturas, las figuras, los ejercicios y sus soluciones. La numeración de temas, secciones, enunciados y ecuaciones es la misma que en los apuntes, de modo que cualquier referencia puede buscarse allí; las que remiten a elementos suprimidos (un ejemplo, una figura, una ecuación de una demostración) conservan su número en los apuntes.

Notación

Símbolo	Significado
$\mathcal{V}$, $K = \lvert\mathcal{V}\rvert$	vocabulario y su tamaño
$x_{<t} = (x_1, \dots, x_{t-1})$	prefijo (contexto) en el paso t
$z \in \mathbb{R}^K$, $\tau > 0$	vector de logits y temperatura (la letra T se reserva para longitudes)
$p_\tau = \mathrm{softmax}(z/\tau)$	distribución con temperatura
k, p, m	parámetros de *top-k*, del muestreo de núcleo y anchura del haz
N, n	parámetros del modelo y longitud del prompt
L, n_{kv}, d_{h}	capas, cabezas de clave y valor, y dimensión de cada cabeza
s	bytes por número (2 en fp16 y bf16, 1 en int8)
B, B_{mem}, P_{pico}	tamaño del lote, ancho de banda de memoria y rendimiento máximo
p, q	en el tema 3, distribución del modelo objetivo y del borrador
γ, β, α, c	tokens propuestos, tasa de aceptación, su valor esperado y coste relativo del borrador
$\mathrm{TV}(p, q)$	distancia de variación total, $\frac{1}{2}\sum_x \lvert p(x) - q(x)\rvert$
b, Δ, Z	bits, paso (escala) y punto cero de un cuantizador
$H = 2XX^\top$	matriz hessiana del error cuadrático de una capa (tema 5)
TTFT, TPOT	tiempo hasta el primer token y tiempo por token de salida

Índice

Sobre este resumen **ii**

1 Decodificación: búsqueda y muestreo **1**
1.1 El problema de la decodificación . . . 1
1.2 Búsqueda voraz y búsqueda en haz . . . 1
1.3 Muestreo con temperatura . . . 2
1.4 Truncamiento: *top-k* y muestreo de núcleo . . . 3

2 El coste de la inferencia: *prefill*, decodificación y caché KV **4**
2.1 Las dos fases de la generación . . . 4
2.2 La caché KV . . . 4
2.3 Intensidad aritmética de la inferencia . . . 5
2.4 El lote: latencia frente a rendimiento . . . 6

3 Decodificación especulativa **7**
3.1 El algoritmo . . . 7
3.2 Muestreo por rechazo modificado . . . 8
3.3 Corrección para secuencias . . . 8
3.4 Rendimiento . . . 8

4 Fundamentos de la cuantización **10**
4.1 Cuantizadores uniformes . . . 10
4.2 Error de cuantización . . . 10
4.3 Granularidad y valores atípicos . . . 11
4.4 Producto de matrices en aritmética entera . . . 12

5 Cuantización de LLM y destilación **13**
5.1 Rasgos atípicos y LLM.int8() . . . 13
5.2 SmoothQuant: migrar la dificultad a los pesos . . . 13
5.3 GPTQ: cuantización con información de segundo orden . . . 14
5.4 AWQ: proteger los canales importantes . . . 15
5.5 QLoRA: cuantizar a 4 bits para ajustar . . . 15
5.6 Destilación de conocimiento . . . 16

6 Servicio a escala: latencia, rendimiento y memoria **17**

6.1 Métricas de servicio 17

6.2 *Batching* estático y continuo 17

6.3 Memoria de la caché KV y PagedAttention 18

Bibliografía **19**

Tema 1

Decodificación: búsqueda y muestreo

Un modelo de lenguaje autorregresivo proporciona, en cada paso, una distribución $P(\cdot \mid x_{<t})$ sobre el vocabulario; la *decodificación* es la regla que convierte esa sucesión de distribuciones en un texto concreto. Este tema presenta las dos familias de reglas: las de *maximización* (búsqueda voraz y búsqueda en haz), que intentan encontrar la secuencia más probable, y las de *muestreo* (temperatura, *top-k* y núcleo), que extraen una secuencia aleatoria de una versión modificada de la distribución. Se siguen el §7.6 y el §13.4 de Jurafsky y Martin [13], el §10.8 de Zhang *et al.* [22], la sección 3 de Holtzman *et al.* [11], el §5.4 de Fan *et al.* [5] y la sección 6 de la tarea 1 de CS336 [2].

1.1 El problema de la decodificación

Sea P un modelo de lenguaje sobre $\mathcal{V}$ con símbolo de fin $\langle . \rangle$ (módulo 1). Por la regla de la cadena, la probabilidad de una continuación $y = (y_1, \ldots, y_T)$ de un prompt x es

$$P(y \mid x) = \prod_{t=1}^{T} P(y_t \mid x, y_{<t}), \qquad \ln P(y \mid x) = \sum_{t=1}^{T} \ln P(y_t \mid x, y_{<t}).$$

Definición 1.1 (Regla de decodificación). Una *regla de decodificación* es un procedimiento que, a partir del prompt x y del acceso a las distribuciones condicionadas $P(\cdot \mid x, y_{<t})$, produce una continuación y, de forma determinista o aleatoria. La *decodificación de máxima probabilidad* (MAP) busca

$$y^* = \arg\max_{y} \; P(y \mid x),$$

donde el máximo se toma sobre las secuencias terminadas en $\langle . \rangle$.

Proposición 1.2. *Con un vocabulario de K tokens, el número de continuaciones de longitud exactamente T es K^T. En consecuencia, la búsqueda exhaustiva del máximo entre las secuencias de longitud T requiere evaluar K^T secuencias.*

Con $K = 5 \cdot 10^4$ y $T = 20$, $K^T \approx 9{,}5 \cdot 10^{93}$: la decodificación MAP exacta es inabordable y se aproxima con búsquedas locales [22, §10.8.2].

1.2 Búsqueda voraz y búsqueda en haz

Definición 1.3 (Búsqueda voraz). La *búsqueda voraz* (*greedy*) elige en cada paso el token más probable, $y_t = \arg\max_{v \in \mathcal{V}} P(v \mid x, y_{<t})$, hasta emitir $\langle . \rangle$ o alcanzar una longitud máxima [13, §7.6.1].

Proposición 1.4. *La búsqueda voraz no encuentra, en general, la secuencia de máxima probabilidad.*

Definición 1.5 (Búsqueda en haz). Sea $m \geq 1$ la *anchura del haz*. La *búsqueda en haz* mantiene un conjunto $\mathcal{H}_t$ de, a lo sumo, m prefijos de longitud t. En el paso $t{+}1$ forma todas las extensiones (h, v) con $h \in \mathcal{H}_t$ y $v \in \mathcal{V}$, las puntúa con $\ln P(h, v \mid x) = \ln P(h \mid x) + \ln P(v \mid x, h)$ y conserva las m de mayor puntuación. Las extensiones que terminan en $\langle . \rangle$ se retiran del haz como candidatas finales; al acabar se devuelve la candidata de mayor puntuación [13, §13.4], [22, §10.8.3].

Proposición 1.6 (Coste y casos extremos). *Para una salida de longitud T y vocabulario de tamaño K, la búsqueda en haz de anchura m evalúa a lo sumo mKT extensiones. Con $m = 1$ coincide con la búsqueda voraz, y con $m \geq K^{T-1}$ no poda ningún prefijo y coincide con la búsqueda exhaustiva entre las secuencias de longitud T.*

Observación 1.7 (Sesgo hacia las secuencias cortas). Cada término $\ln P(y_t \mid x, y_{<t})$ es menor o igual que cero, de modo que la puntuación $\ln P(y \mid x)$ no aumenta al alargar la secuencia. La búsqueda en haz tiende entonces a preferir candidatas cortas. La corrección habitual divide por una potencia de la longitud,

$$\mathrm{punt}(y) = \frac{1}{L^a} \ln P(y \mid x), \qquad L = |y|,\ a \in [0, 1],$$

con $a = 0{,}75$ en el ejemplo de [22, §10.8.3]; con $a = 1$ se obtiene la log-probabilidad media por token de [13, §13.4].

Observación 1.8 (Degeneración). La búsqueda en haz es la regla habitual en traducción automática, donde la salida está muy determinada por la entrada. En la generación abierta produce textos repetitivos: Holtzman *et al.* muestran que el texto humano tiene una probabilidad por token claramente menor que la del texto obtenido por maximización y que las repeticiones se refuerzan a sí mismas, porque cada repetición aumenta la probabilidad de la siguiente [11, §4.3]. Por eso los asistentes conversacionales muestrean.

1.3 Muestreo con temperatura

Definición 1.9 (Softmax con temperatura). Sea $z \in \mathbb{R}^K$ el vector de logits y sea $\tau > 0$. La *distribución con temperatura* τ es

$$p_\tau(i) = \frac{e^{z_i/\tau}}{\sum_{j=1}^{K} e^{z_j/\tau}}, \qquad i = 1, \ldots, K.$$

El *muestreo con temperatura* extrae cada token de p_τ [13, §7.6.3], [11, §3.3]. Con $\tau = 1$ se recupera la distribución del modelo.

Proposición 1.10 (Temperatura como potencia). *Sea $p = p_1$ la distribución del modelo. Entonces $p_\tau(i) \propto p(i)^{1/\tau}$. En particular, elevar las probabilidades a una potencia $a > 0$ y renormalizar equivale a muestrear con temperatura $\tau = 1/a$.*

Proposición 1.11 (Límites de la temperatura). *Sea $M = \{i : z_i = \max_j z_j\}$ el conjunto de logits máximos. Entonces*

$$\lim_{\tau \to 0^+} p_\tau(i) = \begin{cases} 1/|M|, & i \in M, \\ 0, & i \notin M, \end{cases} \qquad \lim_{\tau \to \infty} p_\tau(i) = \frac{1}{K}.$$

Si el máximo es único, el límite $\tau \to 0^+$ es la búsqueda voraz.

La temperatura reescala todos los logits por igual y nunca anula un token: la cola de tokens improbables conserva masa positiva para cualquier $\tau > 0$. Los métodos de truncamiento la eliminan.

1.4 Truncamiento: *top-k* y muestreo de núcleo

Definición 1.13 (Distribución truncada). Sea p una distribución sobre $\mathcal{V}$ y sea $S \subseteq \mathcal{V}$ con $p(S) = \sum_{i \in S} p(i) > 0$. La *distribución truncada* a S es $p_S(i) = p(i)/p(S)$ si $i \in S$ y $p_S(i) = 0$ en otro caso.

Definición 1.14 (*Top-k* y núcleo). Sea $p_{(1)} \geq p_{(2)} \geq \cdots \geq p_{(K)}$ la ordenación decreciente de las probabilidades.

(a) El muestreo *top-k* [5, §5.4] trunca p al conjunto $V^{(k)}$ de los k tokens más probables.

(b) El *muestreo de núcleo* o *top-p* [11, §3.1], con umbral $p \in (0, 1]$, trunca al menor conjunto $V^{(p)}$ tal que $\sum_{i \in V^{(p)}} p(i) \geq p$.

Proposición 1.15 (El núcleo está formado por los más probables). *Sea $k^* = \min\{k : \sum_{i=1}^{k} p_{(i)} \geq p\}$. Entonces el tamaño mínimo de un conjunto con masa al menos p es k^*, y el conjunto de los k^* tokens más probables lo alcanza. Es decir, el muestreo de núcleo coincide con* top-k *con $k = k^*$, donde k^* depende de la distribución de cada paso.*

Proposición 1.16 (Distorsión del truncamiento). *Para cualquier S con $p(S) > 0$, la distancia de variación total entre la distribución truncada y la original es la masa eliminada:*

$$\mathrm{TV}(p_S, p) = \tfrac{1}{2} \sum_{i \in \mathcal{V}} |p_S(i) - p(i)| = 1 - p(S).$$

Así, el muestreo de núcleo con umbral p garantiza $\mathrm{TV}(p_S, p) \leq 1 - p$ en cada paso, mientras que *top-k* con k fijo no controla la masa eliminada: si el modelo duda entre muchas opciones razonables, k fijo descarta masa grande; si está seguro, conserva tokens de cola [11, §3.2].

Observación 1.18 (Combinación y uso práctico). Las reglas se componen en el orden de la figura 1.2: los logits se dividen por τ, se aplica la softmax, se trunca con *top-k* y luego con el núcleo, se renormaliza y se muestrea [13, §7.6.4]. Las API públicas de Anthropic, Google y xAI exponen la temperatura y el umbral del núcleo como parámetros de la petición; los valores que usan por defecto sus aplicaciones de chat no son públicos.

Tema 2

El coste de la inferencia: *prefill*, decodificación y caché KV

Generar una respuesta tiene dos fases con perfiles de coste opuestos: el *prefill*, que procesa el prompt completo en paralelo, y la *decodificación*, que produce los tokens uno a uno. Este tema demuestra que la caché de claves y valores (caché KV) evita recalcular el pasado sin alterar la salida, calcula su tamaño y aplica el modelo *roofline* del módulo 17 para explicar por qué la decodificación está limitada por el ancho de banda de memoria y cómo el tamaño del lote intercambia latencia por rendimiento. Se siguen la clase 10 de CS336 [9], la sección 2 de Pope *et al.* [17], la sección 3 de Kwon *et al.* [14] y el artículo de Shazeer sobre la atención multiconsulta [18].

2.1 Las dos fases de la generación

Definición 2.1 (*Prefill* y decodificación). Sea un prompt $x_{1:n}$. La fase de *prefill* ejecuta una pasada del Transformer sobre las n posiciones a la vez, como en el entrenamiento, y obtiene la distribución del primer token nuevo. La fase de *decodificación* ejecuta, para cada token nuevo, una pasada sobre una sola posición, la del último token emitido.

Por el módulo 16, una pasada hacia delante cuesta unas $2N$ operaciones por token y posición, donde N es el número de parámetros (sin contar la atención sobre el contexto, que se trata en el proposición 2.9).

2.2 La caché KV

En un Transformer causal (módulos 13 y 14), sean $h_i^{(\ell)}$ el estado de la posición i a la salida de la capa ℓ, con $h_i^{(0)}$ la incrustación del token x_i y su posición. Cada capa calcula, para cada posición, una consulta, una clave y un valor por cabeza, $q_i = h_i^{(\ell-1)} W_Q$, $k_i = h_i^{(\ell-1)} W_K$, $v_i = h_i^{(\ell-1)} W_V$, y la salida de la atención en t es

$$a_t = \mathrm{softmax}\Big(\frac{q_t\,[k_1;\dots;k_t]^\top}{\sqrt{d_\mathrm{h}}}\Big)\,[v_1;\dots;v_t],$$

seguida de las conexiones residuales, la normalización y el perceptrón multicapa, que actúan posición a posición.

Lema 2.3 (Causalidad de los estados). *Para toda capa ℓ y toda posición i, el estado $h_i^{(\ell)}$ depende solo de $x_{1:i}$.*

Teorema 2.4 (Corrección de la caché KV). *Sea un procedimiento que, en cada paso de decodificación, calcula q, k y v únicamente para la posición nueva t en cada capa, añade (k_t, v_t)*

a una caché que contiene los pares (k_j, v_j), $j < t$, calculados en pasos anteriores, y atiende sobre toda la caché. Entonces sus salidas coinciden exactamente (en aritmética exacta) con las de una pasada completa sobre $x_{1:t}$.

En coma flotante pueden aparecer diferencias de redondeo, porque el orden de las operaciones cambia; la distribución que define el modelo es la misma.

Proposición 2.5 (Pasadas de posición con y sin caché). *Sea un prompt de n tokens del que se generan T tokens. Sin caché, el número total de posiciones procesadas por las capas no atencionales es $Tn + T(T-1)/2$; con caché es $n + T - 1$.*

Proposición 2.6 (Tamaño de la caché KV). *Sea un modelo con L capas, n_{kv} cabezas de clave y valor de dimensión d_{h}, y números de s bytes. La caché ocupa*

$$c_{\text{kv}} = 2\, L\, n_{\text{kv}}\, d_{\text{h}}\, s \quad \textit{bytes por token,}$$

y una secuencia de S tokens ocupa $S\, c_{\text{kv}}$ bytes.

Observación 2.8 (Atención multiconsulta y agrupada). La caché es proporcional a n_{kv}. La atención multiconsulta (MQA) [18] usa $n_{\text{kv}} = 1$ y la agrupada (GQA, módulo 18) un valor intermedio; respecto a la atención multicabeza con n_h cabezas, la caché se divide por n_h/n_{kv}. Pope *et al.* muestran que la atención multiconsulta permite contextos hasta 32 veces más largos con la misma memoria en PaLM 540B [17].

2.3 Intensidad aritmética de la inferencia

Se recuerda del módulo 17 que la intensidad aritmética de una operación es $I = F/Q$ (FLOP por byte transferido) y que, en un dispositivo con rendimiento P_{pico} y ancho de banda B_{mem}, la operación está limitada por la memoria si $I < I^* = P_{\text{pico}}/B_{\text{mem}}$.

Proposición 2.9 (Intensidad de las capas). *Sea un lote de B secuencias en bf16 ($s = 2$) del que se procesan T posiciones nuevas por secuencia, con S posiciones ya presentes en la caché.*

(a) En un perceptrón con tres matrices de $D \times F$ (como el SwiGLU), $F_{\text{op}} = 6BTDF$ y $Q = 4BTD + 4BTF + 6DF$. Si $BT \ll D, F$, la intensidad es $I \approx BT$.

(b) En la atención, con dimensión total D de consultas, claves y valores, $F_{\text{op}} = 4BSTD$ y $Q = 4BSD + 4BTD$, de modo que $I = ST/(S+T)$, independiente de B. En el prefill *($T = S$), $I = S/2$; en la decodificación ($T = 1$), $I = S/(S+1) < 1$.*

La diferencia entre ambas capas es estructural: en el perceptrón todas las secuencias del lote comparten los mismos pesos, que se leen una vez; en la atención cada secuencia tiene su propia caché, de modo que agrupar secuencias no reutiliza datos [9].

Teorema 2.10 (Cota de memoria de la decodificación). *Sea un modelo de N parámetros de s bytes y sea un paso de decodificación con un lote de B secuencias de S tokens de contexto cada una. Si los pesos y la caché se leen al menos una vez por paso, el tiempo del paso cumple*

$$t_{\text{paso}} \;\geq\; \frac{sN + B\, S\, c_{\text{kv}}}{B_{\text{mem}}}.$$

En particular, con $B = 1$ y caché despreciable, la velocidad de generación cumple tokens/s $\leq B_{\text{mem}}/(sN)$.

2.4 El lote: latencia frente a rendimiento

Proposición 2.12 (Compromiso del lote). *En las condiciones del teorema 2.10, supóngase que el tiempo de un paso es exactamente* $t(B) = t_0 + Bt_1$*, con* $t_0 = sN/B_{\text{mem}}$ *y* $t_1 = S\,c_{\text{kv}}/B_{\text{mem}}$*. Entonces la velocidad por usuario* $1/t(B)$ *decrece con* B*, el rendimiento total* $R(B) = B/t(B)$ *crece con* B *y*

$$\lim_{B\to\infty} R(B) = \frac{1}{t_1} = \frac{B_{\text{mem}}}{S\,c_{\text{kv}}}.$$

El lote amortiza la lectura de los pesos, pero no la de la caché. La cuadro 2.1 lo ilustra con Llama 2 7B en fp16 (14 GB de pesos), secuencias de 2000 tokens (1,05 GB de caché cada una) y $B_{\text{mem}} = 2$ TB/s; el límite del rendimiento es $B_{\text{mem}}/(S\,c_{\text{kv}}) \approx 1907$ tokens/s. Además, la memoria total limita B: con 64 secuencias la caché ya ocupa 67 GB.

Lote B	Caché total (GB)	Tiempo por paso (ms)	Tokens/s totales	Tokens/s por usuario
1	1,05	7,52	133	133
8	8,39	11,19	715	89,3
32	33,55	23,78	1346	42,1
64	67,11	40,55	1578	24,7

Cuadro 2.1: Cota de memoria del teorema 2.10 para Llama 2 7B en fp16 con contexto de 2000 tokens y $B_{\text{mem}} = 2$ TB/s.

Observación 2.13 (Cuándo deja de mandar la memoria). Respecto a los pesos, la intensidad de un paso con lote B es $\approx B$ FLOP/byte en bf16 (proposición 2.9), de modo que haría falta $B \approx I^*$ para quedar limitado por el cómputo; la caché, con intensidad menor que 1, lo impide antes. Pope *et al.* cuantifican este compromiso en PaLM 540B: el *prefill* de lotes grandes alcanza un 76 % de utilización de FLOP, mientras que la decodificación con lote pequeño logra una latencia de 29 ms por token con pesos en int8 [17]. Todo lo que reduzca bytes acelera la decodificación: menos cabezas de clave y valor, cuantización (temas 4 y 5) y más tokens por lectura de pesos (tema 3).

Tema 3

Decodificación especulativa

El tema 2 mostró que un paso de decodificación está limitado por la lectura de los pesos: procesar unas pocas posiciones más en la misma pasada apenas cuesta tiempo. La *decodificación especulativa* aprovecha esta asimetría entre generar y verificar. Un modelo pequeño (el *borrador*) propone varios tokens y el modelo grande (el *objetivo*) los verifica todos en una sola pasada con una regla de aceptación y rechazo que preserva *exactamente* su distribución. Se siguen el artículo de Leviathan, Kalman y Matias [15] (secciones 2 y 3 y apéndice A.1), el de Chen *et al.* [1], que propuso el mismo algoritmo de forma independiente con el nombre de *speculative sampling*, y la clase 10 de CS336 [9].

3.1 El algoritmo

En todo el tema, $p(\cdot \mid x_{<t})$ es la distribución del modelo objetivo y $q(\cdot \mid x_{<t})$ la del borrador, ambas ya modificadas por la regla de muestreo que se desee (temperatura, núcleo, etc.). Cuando el contexto está fijado se escribe simplemente $p(x)$ y $q(x)$.

Definición 3.1 (Ronda de decodificación especulativa). Sea $\gamma \geq 1$ un entero y sea $x_{<t}$ el prefijo actual. Una *ronda* consiste en:

1. El borrador genera $\tilde{x}_1, \ldots, \tilde{x}_\gamma$ de forma autorregresiva, con $\tilde{x}_i \sim q(\cdot \mid x_{<t}, \tilde{x}_{<i})$, y guarda esas probabilidades.
2. El modelo objetivo procesa en una sola pasada el prefijo y los γ tokens propuestos, y obtiene $p(\cdot \mid x_{<t}, \tilde{x}_{<i})$ para $i = 1, \ldots, \gamma + 1$.
3. Para $i = 1, \ldots, \gamma$, en orden, se acepta $\tilde{x}_i$ con probabilidad $\min\big(1,\, p(\tilde{x}_i)/q(\tilde{x}_i)\big)$, con ambas distribuciones condicionadas al mismo prefijo.
4. En el primer rechazo, en la posición i, se emite un token de la *distribución residual*

$$r(x) = \frac{\max\big(0,\, p(x) - q(x)\big)}{\sum_y \max\big(0,\, p(y) - q(y)\big)}$$

   y la ronda termina.
5. Si se aceptan los γ tokens, se emite además un token de $p(\cdot \mid x_{<t}, \tilde{x}_{1:\gamma})$, que ya se calculó en el paso 2.

Cada ronda emite entre 1 y $\gamma + 1$ tokens con una sola pasada del modelo objetivo. Esa pasada procesa $\gamma + 1$ posiciones, como un pequeño *prefill*: está limitada por la lectura de los pesos y cuesta casi lo mismo que un paso de decodificación (teorema 2.10).

3.2 Muestreo por rechazo modificado

Se analiza primero un solo token. Se propone $\tilde{x} \sim q$, que se acepta con probabilidad $\min(1, p(\tilde{x})/q(\tilde{x}))$; si se rechaza, se emite $x' \sim r$.

Definición 3.2 (Tasa de aceptación). La *tasa de aceptación* es la probabilidad de aceptar la propuesta [15, definición 3.1]:

$$\beta = \sum_x q(x) \min\Big(1, \frac{p(x)}{q(x)}\Big) = \sum_x \min\big(p(x), q(x)\big).$$

Lema 3.3 (Normalización del residuo). $\sum_x \max\big(0, p(x) - q(x)\big) = 1 - \beta$. *En consecuencia, r está bien definida si $\beta < 1$, y $\beta = 1$ si y solo si $p = q$.*

Teorema 3.4 (Preservación de la distribución). *El token emitido por el muestreo por rechazo modificado tiene distribución exactamente p.*

Proposición 3.5 (Aceptación y variación total). $1 - \beta = \mathrm{TV}(p, q)$, *de modo que la tasa de rechazo es la distancia de variación total entre el objetivo y el borrador [15, lema 3.3 y teorema 3.5].*

La demostración se pide en el ejercicio 3.5. Un borrador es bueno exactamente en la medida en que su distribución está próxima, en variación total, a la del objetivo; por eso se suele obtener por destilación del objetivo [9] (tema 5).

3.3 Corrección para secuencias

Teorema 3.7 (Preservación en secuencias). *Sea un procedimiento que ejecuta rondas sucesivas (definición 3.1) a partir de un prompt, cada una sobre el prefijo que dejó la anterior. Para todo T, los T primeros tokens emitidos tienen la misma distribución conjunta que T pasos de muestreo autorregresivo de p.*

Corolario 3.8 (Versión voraz). *Si p y q son distribuciones degeneradas en sus respectivos* $\arg\max$ *(búsqueda voraz), la regla de aceptación acepta $\tilde{x}_i$ si y solo si coincide con el* $\arg\max$ *del objetivo, y en caso contrario emite ese* $\arg\max$. *La salida es idéntica a la búsqueda voraz con el objetivo.*

3.4 Rendimiento

Definición 3.9 (Aceptación media y coste relativo). Se supone que las tasas de aceptación de las sucesivas propuestas son variables independientes e idénticamente distribuidas, de media $\alpha = \mathbb{E}[\beta]$ [15, §3.1]. Se llama *coste relativo* c al cociente entre el tiempo de una pasada del borrador y el de una pasada del objetivo [15, definición 3.7].

Teorema 3.10 (Tokens esperados por ronda). *Con aceptaciones independientes de probabilidad $\alpha < 1$, el número N_{r} de tokens emitidos en una ronda cumple*

$$\mathbb{E}[N_{\mathrm{r}}] = \sum_{j=0}^{\gamma} \alpha^j = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}.$$

Teorema 3.11 (Factor de mejora). *En las condiciones anteriores, si el tiempo de una ronda es el de γ pasadas del borrador más una del objetivo, el cociente entre el tiempo esperado por token sin y con decodificación especulativa es*

$$f(\gamma) = \frac{1 - \alpha^{\gamma+1}}{(1-\alpha)(\gamma c + 1)}.$$

Corolario 3.12. *Si $\alpha > c$, existe γ con $f(\gamma) > 1$; en concreto, $f(1) = (1+\alpha)/(1+c) > 1$ [15, corolario 3.9].*

Observación 3.14 (Resultados publicados). Leviathan *et al.* obtienen aceleraciones de 2 a 3 veces en T5-XXL con salidas idénticas [15], y Chen *et al.*, de 2 a 2,5 veces en Chinchilla 70B con un borrador de 4000 millones de parámetros [1]. La clase 10 de CS336 describe extensiones en las que el borrador es una pequeña red añadida al propio objetivo [9]. Si Claude, Gemini o Grok emplean decodificación especulativa en su servicio no es información pública.

Tema 4

Fundamentos de la cuantización

Como la decodificación está limitada por la memoria (tema 2), guardar los pesos con menos bits acelera casi en proporción a la reducción de bytes. *Cuantizar* es representar números reales con un conjunto finito de niveles, normalmente enteros de b bits acompañados de una escala. Este tema define los cuantizadores uniformes afín y simétrico, acota su error, deduce la relación entre bits y relación señal-ruido, analiza la granularidad de las escalas y el efecto de los valores atípicos, y muestra cómo se realiza un producto de matrices en aritmética entera. Se siguen las clases 5 y 6 («Quantization») de MIT 6.5940 [8], el estudio de Gray y Neuhoff [7], la sección 2 de Jacob *et al.* [12], la sección 2 de Xiao *et al.* [20] y el §2.1 de Dettmers *et al.* [3].

4.1 Cuantizadores uniformes

En todo el tema, $\operatorname{round}(u)$ denota el redondeo al entero más próximo con los empates resueltos lejos de cero. Es una función impar, $\operatorname{round}(-u) = -\operatorname{round}(u)$, y cumple $|\operatorname{round}(u) - u| \le \frac{1}{2}$ y $\operatorname{round}(u + n) = \operatorname{round}(u) + n$ para todo entero n.

Definición 4.1 (Cuantizador afín). Sean $b \ge 1$ bits, un *paso* o *escala* $\Delta > 0$ y un *punto cero* entero $Z \in \{0, \dots, 2^b - 1\}$. El *cuantizador afín* asigna a $x \in \mathbb{R}$ el entero

$$Q(x) = \operatorname{clip}\big(\operatorname{round}(x/\Delta) + Z,\ 0,\ 2^b - 1\big),$$

y la *descuantización* es $\hat{x} = \Delta\,(Q(x) - Z)$. Para un grupo de valores con rango $[x_{\min}, x_{\max}]$, con $x_{\min} \le 0 \le x_{\max}$, la calibración por mínimo y máximo toma

$$\Delta = \frac{x_{\max} - x_{\min}}{2^b - 1}, \qquad Z = \operatorname{round}\Big(-\frac{x_{\min}}{\Delta}\Big).$$

Definición 4.2 (Cuantizador simétrico). El *cuantizador simétrico* de b bits usa $Z = 0$, los enteros $\{-(2^{b-1} - 1), \dots, 2^{b-1} - 1\}$ y la escala $\Delta = \max_i |x_i| / (2^{b-1} - 1)$, llamada de *máximo absoluto* (*absmax*). En int8 los enteros van de -127 a 127; LLM.int8() escribe la misma regla como multiplicación por $s_x = 127/\|x\|_\infty = 1/\Delta$ [3, §2.1].

El cuantizador simétrico es más sencillo y es el habitual para los pesos, cuya distribución está centrada en cero; el afín aprovecha mejor los niveles cuando el rango es asimétrico, como en las activaciones posteriores a una ReLU [8].

4.2 Error de cuantización

Teorema 4.3 (Cota del error). *Sea un cuantizador afín calibrado por mínimo y máximo (definición 4.1). Para todo $x \in [x_{\min}, x_{\max}]$, el recorte no actúa y*

$$|\hat{x} - x| \le \frac{\Delta}{2}.$$

*Además, $x = 0$ se representa sin error. Para x fuera del rango, el recorte puede actuar y el error no está acotado (*saturación*).*

La representación exacta del cero importa en la práctica: el relleno con ceros y las activaciones nulas no introducen error [12].

Proposición 4.5 (Modelo de ruido uniforme). *Si el error $e = \hat{x} - x$ se modela como una variable uniforme en $[-\Delta/2, \Delta/2]$, entonces $\mathbb{E}[e] = 0$ y $\mathbb{E}[e^2] = \Delta^2/12$.*

El modelo es una aproximación clásica, buena cuando el paso es pequeño frente a la escala de variación de la densidad de x [7]; la demostración se pide en el ejercicio 4.6. Una simulación con $\Delta = 0{,}01$ y 10^6 muestras da $\mathbb{E}[e^2] \approx 8{,}345 \cdot 10^{-6}$, frente a $\Delta^2/12 = 8{,}333 \cdot 10^{-6}$.

Corolario 4.6 (Seis decibelios por bit). *Si el rango está fijo, cada bit adicional divide Δ entre 2 y el error cuadrático medio entre 4, de modo que la relación señal-ruido aumenta en $10 \log_{10} 4 \approx 6{,}02$ dB por bit.*

4.3 Granularidad y valores atípicos

Definición 4.7 (Granularidad). Sea una matriz de pesos $W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$. La cuantización es *por tensor* si usa una sola escala para toda la matriz; *por canal* si usa una escala por cada canal de salida (columna de W), y *por grupos* si divide cada canal en grupos de g pesos consecutivos con una escala cada uno [20, §2]. Para las activaciones se usa además la cuantización *por token*, con una escala por fila.

Proposición 4.8 (Coste de las escalas). *Con pesos de b bits y una escala de b_Δ bits por grupo de g pesos, el almacenamiento medio es $b + b_\Delta/g$ bits por peso.*

Con int4, grupos de $g = 128$ y escalas en fp16, el coste es $4 + 16/128 = 4{,}125$ bits por peso. La cuadro 4.1 traslada estas cifras a la cota de velocidad del teorema 2.10.

Formato	Bytes por peso	Pesos de un modelo de $7 \cdot 10^9$	Cota de tokens/s a 2 TB/s
fp16 / bf16	2	14 GB	143
int8	1	7 GB	286
int4, grupos de 128	0,5156	3,61 GB	554

Cuadro 4.1: Efecto del formato de los pesos en la memoria y en la cota de velocidad con lote 1.

Proposición 4.9 (Efecto de un valor atípico). *En un grupo cuantizado de forma simétrica con b bits, si un valor tiene módulo $M = \max_i |x_i|$, todos los valores con $|x_i| < \Delta/2 = M/\big(2(2^{b-1} - 1)\big)$ se cuantizan a cero, y la cota de error del grupo es proporcional a M.*

Por ejemplo, un grupo int8 con valores del orden de 0,1 y un atípico $M = 8$ tiene $\Delta = 8/127 = 0{,}063$: los valores pequeños quedan representados con solo uno o dos niveles. Los remedios son grupos más pequeños, tratar los atípicos aparte y transformar la matriz antes de cuantizar; se estudian en el tema 5.

4.4 Producto de matrices en aritmética entera

Proposición 4.10 (Producto cuantizado). *Sean $X \in \mathbb{R}^{B\times n}$ y $W \in \mathbb{R}^{n\times d}$ cuantizadas con $X \approx \Delta_X(Q_X - Z_X)$ (afín, por tensor) y $W \approx \Delta_W Q_W$ (simétrica, por tensor). Entonces*

$$XW \approx \Delta_X \Delta_W \big(Q_X Q_W - Z_X\, \mathbf{1}\mathbf{1}^\top Q_W\big),$$

donde $\mathbf{1} \in \mathbb{R}^B$ y $\mathbf{1} \in \mathbb{R}^n$ son vectores de unos. El producto $Q_X Q_W$ se calcula con enteros y acumuladores de 32 bits, y el término $\mathbf{1}^\top Q_W$ (sumas de las columnas de Q_W) puede precalcularse una vez [12, §2].

Proposición 4.11 (Qué escalas se pueden factorizar). *Sean escalas Δ_i^X por fila de X (por token) y Δ_j^W por columna de W (por canal de salida), con cuantización simétrica. Entonces*

$$(XW)_{ij} \approx \Delta_i^X\, \Delta_j^W \sum_{k=1}^{n} (Q_X)_{ik} (Q_W)_{kj},$$

es decir, la descuantización es un reescalado del producto entero por el producto exterior de las escalas. En cambio, una escala por columna de X (por canal de entrada) depende del índice k que se suma y no se puede sacar del sumatorio.

Esta es la cuantización vectorial de LLM.int8() [3, §3.1], y su limitación explica por qué los atípicos de las activaciones, que se concentran en unos pocos *canales*, son difíciles de tratar con núcleos de cálculo enteros [20, §3].

Observación 4.12 (Cuantización durante o después del entrenamiento). La cuantización *posterior al entrenamiento* (PTQ) calibra escalas y puntos cero con unos pocos datos y no modifica el entrenamiento; la cuantización *consciente del entrenamiento* (QAT) simula el redondeo en la pasada hacia delante para que los pesos se adapten, a cambio de un coste de entrenamiento mucho mayor [9, 8]. Los formatos de coma flotante de 8 bits (fp8) se trataron en el módulo 17. Qué formatos usan Anthropic, Google o xAI para servir sus modelos no es público.

Tema 5

Cuantización de LLM y destilación

Los cuantizadores del tema 4 funcionan bien en redes pequeñas, pero en los LLM aparecen dos dificultades: activaciones con unos pocos canales de magnitud muy grande y la pérdida de precisión al bajar a 4 bits o menos. Este tema estudia los métodos que las resuelven: la descomposición en precisión mixta de LLM.int8(), la migración de la dificultad de SmoothQuant, la corrección de segundo orden de GPTQ, el escalado según las activaciones de AWQ y el formato NF4 de QLoRA. Termina con la destilación de conocimiento, que comprime el modelo cambiando su tamaño en lugar de su formato. Se siguen los artículos originales [3, 20, 6, 16, 4, 10], las clases 6 («Quantization, Part II»), 9 («Knowledge Distillation») y 13 («Efficient LLM Deployment») de MIT 6.5940 [8] y la clase 10 de CS336 [9].

5.1 Rasgos atípicos y LLM.int8()

Dettmers *et al.* observan que, a partir de unos $6{,}7 \cdot 10^9$ parámetros, las activaciones de los Transformer presentan *rasgos atípicos*: unas pocas dimensiones ocultas con valores de magnitud mayor o igual que 6, presentes de forma sistemática en casi todas las capas [3, §4]. Por la proposición 4.9, una escala por token se ve dominada por ellos y destruye la precisión del resto.

Definición 5.1 (Descomposición en precisión mixta). Sean $X \in \mathbb{R}^{B\times n}$ y $W \in \mathbb{R}^{n\times d}$ y sea $\alpha_0 > 0$ un umbral. Sea $O \subseteq \{1,\dots,n\}$ el conjunto de dimensiones internas h en las que alguna entrada de la columna X^h supera α_0 en valor absoluto. *LLM.int8()* calcula

$$XW \approx \sum_{h\in O} X^h W_h \;+\; S \cdot \sum_{h\notin O} Q(X^h)\, Q(W_h),$$

donde X^h es la columna h de X, W_h la fila h de W, el primer término se calcula en fp16 y el segundo en int8 con cuantización vectorial (escala por fila de X y por columna de W, proposición 4.11) y reescalado S [3, §3.2].

Con $\alpha_0 = 6$, los atípicos ocupan del orden del 0,1 % de las dimensiones, de modo que el 99,9 % de los productos se hace en 8 bits; en modelos de hasta $13 \cdot 10^9$ parámetros hay como mucho 7 dimensiones atípicas [3, §3.2].

Proposición 5.2 (Exactitud de la descomposición). *La descomposición de la definición 5.1 es exacta salvo por el error de cuantización del segundo término: $XW = \sum_{h\in O} X^h W_h + \sum_{h\notin O} X^h W_h$.*

5.2 SmoothQuant: migrar la dificultad a los pesos

Los pesos son fáciles de cuantizar y las activaciones no. SmoothQuant traslada parte de la magnitud de cada canal de entrada de las activaciones a la fila correspondiente de los pesos

mediante una transformación exacta [20, §4].

Proposición 5.3 (Equivalencia del suavizado). *Para todo vector $s \in \mathbb{R}^n$ con componentes positivas,* $XW = \big(X\,\mathrm{diag}(s)^{-1}\big)\big(\mathrm{diag}(s)W\big)$.

Definición 5.4 (Factores de suavizado). Sea $\alpha \in [0,1]$ la *intensidad de migración*. SmoothQuant toma, para cada canal j,

$$s_j = \frac{\max_i |X_{ij}|^{\alpha}}{\max_k |W_{jk}|^{1-\alpha}},$$

con estadísticas de X obtenidas en un conjunto de calibración [20, ec. 4].

Proposición 5.5 (Equilibrio con $\alpha = \frac{1}{2}$). *Sean $\hat{X} = X\,\mathrm{diag}(s)^{-1}$ y $\hat{W} = \mathrm{diag}(s)W$, y sean $m_j^X = \max_i |X_{ij}|$ y $m_j^W = \max_k |W_{jk}|$. Con los factores de la definición 5.4,*

$$\max_i |\hat{X}_{ij}| = (m_j^X)^{1-\alpha}(m_j^W)^{1-\alpha}, \qquad \max_k |\hat{W}_{jk}| = (m_j^X)^{\alpha}(m_j^W)^{\alpha}.$$

En particular, con $\alpha = \frac{1}{2}$ ambos máximos valen $\sqrt{m_j^X m_j^W}$.

El factor $\mathrm{diag}(s)^{-1}$ puede incorporarse de antemano a la capa anterior (por ejemplo, a los parámetros de la normalización), de modo que la transformación no cuesta nada en inferencia y permite cuantizar pesos y activaciones a int8. Los autores proponen $\alpha = 0{,}5$ como punto de equilibrio para OPT y BLOOM y otros valores para otros modelos [20, §4].

5.3 GPTQ: cuantización con información de segundo orden

Redondear cada peso a su nivel más próximo (RTN, *round-to-nearest*) ignora que los errores de pesos distintos pueden compensarse en la salida de la capa. GPTQ minimiza directamente el error de salida.

Definición 5.6 (Problema de cuantización por capas). Sea una capa lineal con pesos $W \in \mathbb{R}^{d_{\mathrm{row}} \times d_{\mathrm{col}}}$ y sea $X \in \mathbb{R}^{d_{\mathrm{col}} \times m}$ una matriz con m entradas de calibración. Se busca la matriz cuantizada $\hat{W}$ que minimiza $\|WX - \hat{W}X\|_F^2$ [6, §3, ec. 1].

El objetivo se separa por filas: para una fila w y su cuantizada $\hat{w} = w + \delta$, el error es $\|\delta X\|^2 = \delta X X^{\top}\delta^{\top} = \frac{1}{2}\,\delta H \delta^{\top}$, con $H = 2XX^{\top}$. Al ser la capa lineal, esta forma cuadrática es exacta y no una aproximación de Taylor.

Teorema 5.7 (Actualización óptima tras cuantizar un peso). *Sea H simétrica definida positiva y sea $E(\delta) = \frac{1}{2}\delta H \delta^{\top}$. Si se fija la componente q de la perturbación al error de redondeo, $\delta_q = \mathrm{quant}(w_q) - w_q$, la perturbación de los demás pesos que minimiza E es*

$$\delta = -\frac{w_q - \mathrm{quant}(w_q)}{[H^{-1}]_{qq}}\,(H^{-1})_{q,:}, \qquad \textit{con error mínimo} \qquad E = \frac{\big(w_q - \mathrm{quant}(w_q)\big)^2}{2\,[H^{-1}]_{qq}}.$$

Este resultado es la fórmula del *Optimal Brain Surgeon* que usa el método OBQ [6, §3, ec. 2]. GPTQ lo hace aplicable a modelos de cientos de miles de millones de parámetros con tres ideas [6, §4]: cuantizar las columnas en un orden fijo, igual para todas las filas, en lugar de elegir en cada paso el peso de menor error; aplicar las actualizaciones por bloques de 128 columnas para reducir el tráfico de memoria, y obtener las filas necesarias de H^{-1} con una

factorización de Cholesky, más estable numéricamente, añadiendo a H un pequeño múltiplo de la identidad. Así cuantiza modelos de unos $175 \cdot 10^9$ parámetros a 3 o 4 bits en unas cuatro horas de una GPU, con pérdida de precisión pequeña [6].

5.4 AWQ: proteger los canales importantes

Lin *et al.* observan que conservar en fp16 alrededor del 1 % de los pesos reduce mucho el error de cuantización si esos pesos se eligen por la magnitud de las *activaciones* que los multiplican, y no por la de los propios pesos [16, §3.1]. Como la precisión mixta es incómoda para el hardware, AWQ consigue el mismo efecto con un escalado.

Proposición 5.8 (Efecto del escalado de un canal). *Sea un peso w que multiplica a una entrada x, cuantizado con el cuantizador simétrico $Q(w) = \Delta \operatorname{round}(w/\Delta)$. Si se escala el peso por $s > 1$ y la entrada por $1/s$, el error de la salida es*

$$Q(ws)\,\frac{x}{s} - wx = \frac{\Delta'}{s}\Big(\operatorname{round}\Big(\frac{ws}{\Delta'}\Big) - \frac{ws}{\Delta'}\Big)x,$$

donde Δ' es el paso del grupo tras el escalado. Si $\Delta' \approx \Delta$ y el error de redondeo relativo al paso tiene la misma distribución en ambos casos, el error esperado de ese canal se divide aproximadamente por s [16, §3.2, ecs. 2 y 3].

La hipótesis $\Delta' \approx \Delta$ es razonable si se escala un solo canal y no es el de mayor módulo del grupo; si s es demasiado grande, el paso del grupo aumenta y empeoran los demás canales. AWQ elige $s = s_X^{\alpha}$, con s_X la magnitud media de cada canal de activación y $\alpha \in [0, 1]$ obtenido por búsqueda en un conjunto de calibración [16, §3.2, ec. 5]. El método solo cuantiza pesos y no necesita retropropagación.

5.5 QLoRA: cuantizar a 4 bits para ajustar

QLoRA ajusta un modelo congelado en 4 bits con adaptadores de bajo rango (módulo 20) e introduce dos técnicas de cuantización [4, §3].

Definición 5.9 (NormalFloat de 4 bits). Sea $Q_{\mathcal{N}}$ la función cuantil de la normal estándar. El formato NF4 toma como niveles los valores

$$q_i = \tfrac{1}{2}\Big(Q_{\mathcal{N}}\Big(\frac{i}{2^k+1}\Big) + Q_{\mathcal{N}}\Big(\frac{i+1}{2^k+1}\Big)\Big),$$

con $k = 4$, ajustados para que el 0 se represente exactamente y normalizados al intervalo $[-1, 1]$; cada bloque de 64 pesos se divide por su máximo absoluto antes de cuantizarse [4, §3, ec. 4].

Los pesos entrenados tienen una distribución aproximadamente normal; al situar los niveles en cuantiles de la normal, cada nivel recibe una fracción parecida de los pesos, cosa que no ocurre con una rejilla uniforme.

Proposición 5.10 (Doble cuantización). *Con bloques de 64 pesos y una constante fp32 por bloque, las constantes cuestan $32/64 = 0{,}5$ bits por peso. Si esas constantes se cuantizan a su vez en 8 bits, en bloques de 256 con una constante fp32 por bloque, el coste baja a $8/64 + 32/(64 \cdot 256) \approx 0{,}127$ bits por peso, un ahorro de unos $0{,}373$ bits por peso [4, §3].*

Con estas técnicas y optimizadores paginados, QLoRA ajusta un modelo de $65 \cdot 10^9$ parámetros en una sola GPU de 48 GB [4].

5.6 Destilación de conocimiento

La destilación entrena un modelo pequeño (el *estudiante*) para imitar las distribuciones de salida de uno grande (el *profesor*), que contienen más información que las etiquetas: las probabilidades relativas de las respuestas incorrectas [10, §2].

Definición 5.11 (Pérdida de destilación). Sean $v \in \mathbb{R}^K$ los logits del profesor y $z \in \mathbb{R}^K$ los del estudiante para una misma entrada, y sea $\tau > 0$. Sean $p = \operatorname{softmax}(v/\tau)$ los *objetivos blandos* y $q = \operatorname{softmax}(z/\tau)$. La pérdida de destilación es la entropía cruzada $C = -\sum_i p_i \ln q_i$, que en la práctica se combina con la entropía cruzada respecto a las etiquetas a temperatura 1 [10, §2, ec. 1].

Proposición 5.12 (Gradiente de la pérdida de destilación). $\dfrac{\partial C}{\partial z_i} = \dfrac{1}{\tau}(q_i - p_i)$.

Para temperaturas altas y logits de media nula, el gradiente se aproxima por $(z_i - v_i)/(K\tau^2)$: destilar equivale entonces a igualar logits por mínimos cuadrados, y los gradientes de los objetivos blandos escalan como $1/\tau^2$, por lo que se multiplican por τ^2 al combinarlos con los de las etiquetas [10, §2.1]; la demostración se pide en el ejercicio 5.7.

Observación 5.13 (Destilación e inferencia). La clase 10 de CS336 describe una receta para obtener modelos más baratos: definir una arquitectura menor (por ejemplo, podando capas, cabezas o dimensiones del original), inicializarla con los pesos del original y reparar la pérdida de calidad por destilación [9]. La destilación también se usa para obtener borradores próximos al objetivo en la decodificación especulativa (tema 3). Si los modelos de las familias Claude, Gemini o Grok que se ofrecen en tamaños menores se obtienen por destilación es, en general, información no pública salvo que el fabricante lo haya declarado.

Tema 6

Servicio a escala: latencia, rendimiento y memoria

Un servidor de inferencia recibe peticiones que llegan en momentos distintos, con prompts y respuestas de longitudes muy diferentes. Este tema define las métricas con las que se mide (tiempo hasta el primer token, tiempo por token y rendimiento), analiza por qué el *batching* estático desperdicia el acelerador y cómo lo resuelve el *batching* continuo, y estudia la gestión paginada de la caché KV, que elimina la fragmentación y permite compartir prefijos. Se siguen la clase 10 de CS336 [9], el artículo de Orca de Yu *et al.* [21], las secciones 3 y 4 de Kwon *et al.* [14] y la sección 2 de Pope *et al.* [17].

6.1 Métricas de servicio

Definición 6.1 (Latencia y rendimiento). Para una petición, el *tiempo hasta el primer token* (TTFT) es el intervalo entre su llegada y la emisión del primer token de la respuesta; incluye la espera en cola y el *prefill*. El *tiempo por token de salida* (TPOT) es el intervalo medio entre tokens sucesivos, y su inverso es la velocidad que percibe el usuario. El *rendimiento* (*throughput*) del sistema es el número total de tokens generados por segundo para todas las peticiones.

Proposición 6.2 (Latencia de una respuesta). *Una respuesta de n_{out} tokens se completa en* $\text{TTFT} + (n_{\text{out}} - 1)\,\text{TPOT}$.

Las dos métricas dependen de fases distintas: el TTFT del *prefill*, limitado por el cómputo, y el TPOT de la decodificación, limitada por la memoria (tema 2). Por eso conviene procesar el *prefill* con lotes pequeños, para reducir el TTFT, y la decodificación con lotes grandes, para aumentar el rendimiento [9]. Además, replicar el modelo en M grupos de aceleradores multiplica el rendimiento por M sin cambiar la latencia, mientras que repartir un modelo entre aceleradores (módulo 17) reduce la latencia a cambio de comunicación [9, 17].

6.2 *Batching* estático y continuo

Definición 6.4 (*Batching* estático). En el *batching* estático, el servidor forma un lote de B peticiones y lo ejecuta hasta que termina la más larga; solo entonces admite peticiones nuevas.

Proposición 6.5 (Utilización del lote estático). *Si las respuestas del lote tienen longitudes $\ell_1, \ldots, \ell_B$, la fracción de ranuras de cómputo útiles durante la decodificación es*

$$U = \frac{\sum_{i=1}^{B} \ell_i}{B \max_i \ell_i}.$$

Definición 6.6 (*Batching* continuo). En el *batching* continuo, o planificación por iteración, el servidor decide la composición del lote en cada paso de decodificación: cuando una secuencia termina, su ranura se asigna inmediatamente a una petición en espera [21]. Como las secuencias tienen longitudes distintas, Orca aplica el *batching selectivo*: las operaciones que actúan posición a posición (proyecciones y perceptrón) se ejecutan sobre la concatenación de todas las posiciones del lote, y la atención se calcula por separado para cada secuencia [21, 9].

Proposición 6.7 (Utilización del *batching* continuo). *Si en todo paso hay al menos tantas peticiones en espera como ranuras liberadas, el* batching *continuo mantiene ocupadas todas las ranuras de decodificación.*

La proposición ignora el coste del *prefill* de las peticiones que entran, que se intercala con la decodificación de las demás; su planificación es un problema propio de los sistemas de servicio.

6.3 Memoria de la caché KV y PagedAttention

Antes de vLLM, los servidores reservaban para cada petición un bloque contiguo de caché KV del tamaño máximo posible de la respuesta. Esa reserva produce *fragmentación interna* (posiciones reservadas que nunca se usan) y *fragmentación externa* (huecos entre reservas de tamaños distintos); en los sistemas que Kwon *et al.* analizan, solo entre el 20,4 % y el 38,2 % de la memoria de la caché guarda estados de tokens [14, §1 y §3.1].

Definición 6.8 (PagedAttention). *PagedAttention* divide la caché KV de cada secuencia en *bloques* de tamaño fijo de b_{blk} tokens, que no tienen por qué ser contiguos en memoria. Cada secuencia tiene una *tabla de bloques* que traduce cada bloque lógico (posiciones $jb_{\text{blk}}, \dots, (j+1)b_{\text{blk}} - 1$) a un bloque físico. Los bloques se asignan cuando se llenan los anteriores, y el núcleo de atención lee la caché siguiendo la tabla [14, §4.1 y §4.2].

La analogía es la memoria virtual paginada de un sistema operativo: la secuencia ve un espacio lógico contiguo, formado por páginas físicas dispersas, y la tabla de bloques hace de tabla de páginas.

Proposición 6.9 (Desperdicio acotado). *Con bloques de b_{blk} tokens, una secuencia de $\ell \geq 1$ tokens ocupa $\lceil \ell / b_{\text{blk}} \rceil$ bloques y desperdicia $b_{\text{blk}} \lceil \ell / b_{\text{blk}} \rceil - \ell \leq b_{\text{blk}} - 1$ posiciones. Como todos los bloques tienen el mismo tamaño, no hay fragmentación externa: cualquier bloque libre sirve a cualquier secuencia.*

Proposición 6.11 (Compartición de prefijos). *Sean n_s secuencias que comparten un prompt de P tokens. Con compartición de bloques y* copia en escritura*, el prompt ocupa $\lfloor P / b_{\text{blk}} \rfloor$ bloques completos compartidos más, si P no es múltiplo de b_{blk}, una copia del bloque parcial por secuencia, en lugar de $n_s \lceil P / b_{\text{blk}} \rceil$ bloques.*

Este mecanismo sirve para el muestreo de varias respuestas por prompt, para la búsqueda en haz y para los prompts de sistema comunes. Las API públicas de Anthropic y de Google ofrecen una caché de prefijos del prompt (*prompt caching* y *context caching*) como función documentada; su implementación interna no es pública.

Bibliografía

[1] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre y J. Jumper. Accelerating large language model decoding with speculative sampling. Prepublicación, DeepMind, 2023. https://arxiv.org/abs/2302.01318

[2] Equipo docente de CS336. CS336: Language Modeling from Scratch. Tarea 1, «Basics», versión 26.0.3. Stanford University, primavera de 2026. Sección 6, «Generating text». https://github.com/stanford-cs336/assignment1-basics

[3] T. Dettmers, M. Lewis, Y. Belkada y L. Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for Transformers at scale. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. https://arxiv.org/abs/2208.07339

[4] T. Dettmers, A. Pagnoni, A. Holtzman y L. Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. https://arxiv.org/abs/2305.14314

[5] A. Fan, M. Lewis e Y. Dauphin. Hierarchical neural story generation. En *Proceedings of ACL*, 2018. https://arxiv.org/abs/1805.04833

[6] E. Frantar, S. Ashkboos, T. Hoefler y D. Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained Transformers. En *International Conference on Learning Representations (ICLR)*, 2023. https://arxiv.org/abs/2210.17323

[7] R. M. Gray y D. L. Neuhoff. Quantization. *IEEE Transactions on Information Theory*, 44(6):2325–2383, 1998.

[8] S. Han. 6.5940: TinyML and Efficient Deep Learning Computing. Massachusetts Institute of Technology, otoño de 2024. Clases 5 y 6 («Quantization»), 9 («Knowledge Distillation») y 13 («Efficient LLM Deployment»). https://hanlab.mit.edu/courses/2024-fall-65940

[9] T. Hashimoto y P. Liang. CS336: Language Modeling from Scratch. Stanford University, primavera de 2026. Clase 10, «Inference». https://github.com/stanford-cs336/lectures

[10] G. Hinton, O. Vinyals y J. Dean. Distilling the knowledge in a neural network. Prepublicación, 2015 (*NIPS Deep Learning Workshop*, 2014). https://arxiv.org/abs/1503.02531

[11] A. Holtzman, J. Buys, L. Du, M. Forbes e Y. Choi. The curious case of neural text degeneration. En *International Conference on Learning Representations (ICLR)*, 2020. https://arxiv.org/abs/1904.09751

[12] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam y D. Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. En *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018. https://arxiv.org/abs/1712.05877

[13] D. Jurafsky y J. H. Martin. *Speech and Language Processing*, 3.ª ed., borrador del 19 de agosto de 2026. Cap. 7, «Transformers and Pretraining» (§7.6 «Decoding»), y cap. 13, «Machine Translation» (§13.4 «Decoding in MT: Beam Search»). https://web.stanford.edu/~jurafsky/slp3/

[14] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang e I. Stoica. Efficient memory management for large language model serving with PagedAttention. En *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023. https://arxiv.org/abs/2309.06180

[15] Y. Leviathan, M. Kalman e Y. Matias. Fast inference from Transformers via speculative decoding. En *Proceedings of the 40th International Conference on Machine Learning (ICML)*, PMLR 202, 2023. https://arxiv.org/abs/2211.17192

[16] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan y S. Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. En *Proceedings of Machine Learning and Systems (MLSys)*, 2024. https://arxiv.org/abs/2306.00978

[17] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, A. Levskaya, J. Heek, K. Xiao, S. Agrawal y J. Dean. Efficiently scaling Transformer inference. En *Proceedings of Machine Learning and Systems (MLSys)*, 2023. https://arxiv.org/abs/2211.05102

[18] N. Shazeer. Fast Transformer decoding: One write-head is all you need. Prepublicación, 2019. https://arxiv.org/abs/1911.02150

[19] University of Washington. CSE 493G1/599G1: Deep Learning for Computer Vision. Primavera de 2025. Sección (*recitation*) 4, «Quantization». https://courses.cs.washington.edu/courses/cse493g1/25sp/schedule/recitations/rec4.pdf

[20] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth y S. Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. En *Proceedings of the 40th International Conference on Machine Learning (ICML)*, PMLR 202, 2023. https://arxiv.org/abs/2211.10438

[21] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim y B.-G. Chun. Orca: A distributed serving system for Transformer-based generative models. En *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022. https://www.usenix.org/system/files/osdi22-yu.pdf

[22] A. Zhang, Z. C. Lipton, M. Li y A. J. Smola. *Dive into Deep Learning*, versión 1.0.3. Cambridge University Press, 2023. §4.1 «Softmax Regression» y §10.8 «Beam Search». https://d2l.ai/